Operational Identity: A Finite Audit of Declared and Implemented Rules of Sameness

Denise M. Case

Department of Computer Science and Information Systems
Northwest Missouri State University, Maryville, MO, USA

Abstract

A record system declares when two records refer to the same entity, occurrence, scope, or rule. Its disclosed implementation mechanisms induce a corresponding operational identity relation. The declared and implemented relations may diverge systematically without producing a provenance gap or detectable contradiction. A system can apply, consistently and with every record individually correct, a rule of sameness that no artifact declares. This paper formalizes that implemented relation. A declared identity regime partitions a finite record domain into co-reference classes; a disclosed mechanism, through its typed identity-relevant outcomes, induces an *operational identity partition* of the same domain. The audit compares these partitions in the refinement lattice. A mechanism is *faithful* when the declared partition refines the operational partition, so no declared class is split. A *divergence witness* is a pair the declaration merges and the mechanism separates; such witnesses are decidable by pair enumeration. When an imported sibling basis also splits a declared class, local comparison with its partition yields sibling-aligned, sub-sibling, super-sibling, or sibling-incomparable divergence. This classification reports only the relationship; it does not identify the basis carried by the mechanism. Global equality of the operational and sibling partitions is defined separately as *regime*

substitution and does not follow from sibling alignment. A version field incremented on every textual edit inhabits the sub-sibling case by splitting declared classes more finely than either imported basis. The audit is three-valued and relative to the disclosed artifacts, evaluated surfaces, and identified uses; each boundary has a finite refuting witness. A passing verdict is non-monotone because extending the transformation history can merge declared classes and create a witness among records already examined.

Keywords: identity; audit; co-reference; partitions; record systems; provenance; entity resolution; legal alignment

1 Introduction

Modern accountability systems increasingly operate under persistent disagreement. Participants may contest what occurred, why it occurred, which rules apply, who bears responsibility, or what consequences should follow. Stable reference keeps competing claims anchored to the same subject matter throughout the dispute.

The record infrastructure supporting those systems must preserve that reference as records are transformed (e.g. amended, forked, refined, decomposed, aggregated, reclassified) and exchanged across institutional boundaries. It must also distinguish the identity commitments needed for shared reference from the causal, legal, and normative conclusions that remain open to dispute. A record system that collapses those layers may make continued reference depend on acceptance of the interpretation under examination.

Neutral Substrates (Case 2026a) establishes the neutrality-by-design constraint for this setting. The foundational layer carries stable reference and permitted attribution, while causal and normative interpretation remains in attributed extensions. *Referential Regimes* (Case 2026b) identifies regime-relative identity bases and transformation classifications needed to preserve reference under ordinary change. Together, the papers describe what a system should declare; this paper examines what deployed systems do.

1.1 Two Answers to One Question

A record system answers the question *do these two records refer to the same thing?* twice.

It answers once in its *declaration*. The declaration states what must remain fixed for a referent to remain the same, and a transformation history then determines which records co-refer. That answer partitions the records into co-reference classes.

It answers again in its *behavior*. An application assigns referent identifiers, resolves references, merges or forks records, admits or rejects operations, and selects transformation rules. Those behaviors also sort records into groups that the system treats as the same and groups that it treats as different. They partition the same records a second way.

The two partitions may differ. When they do, the system operates under a rule of sameness that no artifact states, and may do so with perfect internal consistency. A schema may declare content-fixed identity for a rule. A workflow may decide that a structural revision creates a new rule. An application may use a version field to refuse co-reference between two texts. Each component behaves consistently. The combined system applies an identity model that no single artifact declares.

This failure may be invisible to ordinary consistency checking. A system that consistently applies an undeclared sameness rule issues no contradictory claims. Its provenance graph may record every revision accurately. Its audit log may record every identifier assignment faithfully. The missing fact is the relation between two partitions.

1.2 The Operational Identity Partition

This paper names the second partition and makes it computable.

A disclosed implementation mechanism is admitted as an *audit surface* when its value determines a typed, finite set of identity-relevant outcomes: referent-identifier assignment, referent co-reference, referent persistence, transformation classification, applicability selection, or operation admissibility. The requirement that those outcomes be determined by the mechanism's value is what licenses reading a difference in treatment as controlled by it. The requirement that the mechanism be

drawn from a disclosed registry is what prevents the auditor from manufacturing surfaces at will.

Grouping records by their identity-relevant outcomes yields the *operational identity partition* induced by that surface. The declared regime yields the declared partition. Both are partitions of the same finite record domain, and the audit is a comparison of the two in the refinement lattice.

Faithfulness is one direction of that comparison: the declared partition refines the operational one, so the surface never splits a declared co-reference class. Its refutation is a finite object. A *divergence witness* is a pair (r, s) with

$$r \sim_\tau s \qquad \text{and} \qquad \mathsf{IT}_d(r) \neq \mathsf{IT}_d(s).$$

The declaration merges the pair. The surface separates it. The surface therefore carries an identity distinction the declaration does not.

1.3 Classifying the Divergence

Six of the nine imported regimes lie on a sibling axis, where a second identity basis is available for the same carrier kind. When that sibling also splits at least one examined declared class, the divergence can be classified. Restricting both the operational relation and the sibling relation to the declared co-reference classes yields two equivalence relations that fall into one of four cases: aligned with the sibling, strictly finer than the sibling, strictly coarser than the sibling, or incomparable with the sibling.

The comparison is local to the declared classes, and that locality bounds what alignment can establish. Sibling alignment says the surface splits declared classes as the sibling does. It does not say the surface implements the sibling regime, because the two may still disagree on pairs the declaration separates. The stronger global condition, that the operational partition equals the sibling partition on the whole domain, is defined separately as *regime substitution*, and Proposition 4.14 shows sibling alignment does not imply it. Regime substitution is the condition an earlier formulation of this result named the hidden identity regime.

The worked version-field example occupies the sub-sibling case rather than the aligned one. A version field incremented on every textual edit splits declared co-

reference classes strictly more finely than the sibling. Section 6 exhibits that case, and Proposition 4.15 proves it cannot be told from sibling alignment by any single witness pair. Diagnosing it as a hidden sibling regime would prescribe the wrong repair.

The audit extends to the three regimes with no sibling. Faithfulness and divergence are defined for every regime in the inventory. A divergence is unpositioned when no sibling is available or when the sibling makes no additional distinction within the examined declared classes.

1.4 Contributions

This paper makes five contributions.

First, it defines the operational identity partition, the rule of sameness induced by an examined implementation surface, as a computable object over a disclosed mechanism registry.

Second, it defines faithfulness as a refinement relation between the declared and operational partitions, gives the surface-level asymmetry under which a split refutes faithfulness while a merge does not, and supplies a finite divergence witness that refutes it.

Third, where the sibling is informative on the examined domain, it classifies divergence against the sibling partition in four local lattice cases, distinguishes that classification from the global condition under which the surface actually implements the sibling regime, and gives a finite version-field construction occupying the sub-sibling case.

Fourth, it proves that a passing verdict is not monotone under extension of the transformation history alone. Because declared co-reference is a transitive closure, adding history to a fixed record domain can merge previously separate declared classes and create a witness among records the audit already examined, with no change to the surface and no change to those records.

Fifth, it makes the audit disclosure-relative in three respects and says so. The artifacts disclosed, the surfaces evaluated over them, and the identity-relevant uses identified for each are all disclosure artifacts, each carries a completeness claim the procedure cannot discharge, and each has a witness object whose later discovery

refutes that claim.

The paper introduces no record architecture and requires no reference checker. An auditor may construct the required finite inputs from provenance records, schemas, workflows, configuration, source inspection, runtime traces, or differential tests.

Section 2 states the imported interface and the preconditions. Section 3 defines surfaces and the operational identity partition. Section 4 compares the two partitions. Section 5 gives decidability, the verdict, and non-monotonicity. Section 6 works a legal-alignment example. Section 7 states the disclosure boundary. Section 8 situates the result. Section 9 states its limits, and Section 10 concludes.

2 Imported Interface and Preconditions

This paper uses a small interface imported from *Neutral Substrates* and *Referential Regimes*. The earlier papers supply the neutrality argument, carrier inventory, transformation basis, identity-basis construction, regime assignments, and nine-regime lower bound. The present result needs regime-relative classification, induced co-reference, and the three sibling axes.

2.1 Regimes and Declared Co-reference

Definition 2.1 (Identity Regime)**.** An *identity regime* ρ consists of an identity basis and a classification function

$$\mathrm{Class}_\rho : \mathcal{F} \longrightarrow \{\mathsf{PRS}, \mathsf{BRK}, \mathsf{NEU}, \mathsf{N/A}\}.$$

The identity basis states what must remain fixed for a referent to remain the same. The classification function states whether each transformation family preserves identity, breaks identity, is identity-neutral, or is inapplicable under that basis.

Let a finite family-labeled transformation history be

$$H = \langle (f_i, x_i, y_i) \rangle_{i=1}^{n},$$

where $f_i \in \mathcal{F}$ is the transformation family and $x_i, y_i \in R$ are its source and result records. The family label is regime-independent. The classification changes with the

regime. For histories H and H^*, write $H \preceq H^*$ when H is a subsequence of H^*, so H^* extends H without removing or changing any transformation already recorded.

Definition 2.2 (Non-breaking Edge Set)**.** For a regime ρ, define

$$E_\rho = \{(x_i, y_i) \mid \mathrm{Class}_\rho(f_i) \in \{\mathsf{PRS}, \mathsf{NEU}\}\}.$$

Remark 2.3 (Why Neutral Families Contribute Edges). A family classified PRS under ρ acts on the ρ-identity basis and leaves it fixed. A family classified NEU under ρ does not act on that basis at all. Neither can separate referents under ρ, so both yield non-breaking edges, for different reasons. Only BRK severs the relation. A family classified N/A does not apply to the carrier kind, and Definition 2.10 excludes it from a well-formed history for records of that kind.

Definition 2.4 (Declared Identity Relation and Partition)**.** The declared identity relation $\sim_\rho$ is the equivalence relation generated by E_ρ. Its quotient

$$\pi_\rho = R \,/\!\sim_\rho$$

is the *declared identity partition* of the record domain R. Each block is a set of records the regime treats as representations of one referent.

Definition 2.5 (Core Sibling Axes)**.** The core sibling axes are

$$\mathcal{A} = \big\{\{\mathrm{LOC}, \mathrm{OBJ}\}, \{\text{SCOPE-E}, \text{SCOPE-S}\}, \{\text{RULE-C}, \text{RULE-S}\}\big\}.$$

Each pair supplies two imported identity bases for one carrier kind: locus-fixed and object-fixed identity for plain referents, extension-fixed and structure-fixed identity for scopes, and content-fixed and structure-fixed identity for rules.

Definition 2.6 (Sibling Function)**.** For a regime τ, define

$$\mathrm{Sib}(\tau) = \begin{cases} \tau', & \text{when } \{\tau, \tau'\} \in \mathcal{A}; \\ \varnothing, & \text{when } \tau \text{ lies on no imported sibling axis.} \end{cases}$$

Definition 2.7 (Imported Referential-Regime Interface)**.** The *imported referential-regime interface* is

$$\mathfrak{R} = \big(\mathcal{F}, \mathcal{P}, \{\mathrm{Class}_\rho\}_{\rho \in \mathcal{P}}, \mathrm{Sib}\big),$$

where:

(a) $\mathcal{F}$ is the finite imported transformation-family set;

(b) $\mathcal{P}$ is the finite imported regime inventory;

(c) each

$$\mathrm{Class}_\rho : \mathcal{F} \longrightarrow \{\mathsf{PRS}, \mathsf{BRK}, \mathsf{NEU}, \mathsf{N/A}\}$$

is decidable; and

(d) Sib is the imported partial sibling function.

Remark 2.8 (Interface Revision). Every declared partition, sibling partition, divergence classification, and audit verdict in this paper is relative to a fixed imported interface $\mathfrak{R}$.

Adding a transformation family, adding an identity regime, revising a classification function, or revising the sibling relation produces a new interface version. The declared and sibling partitions must then be recomputed, the finite audit must be rerun, and inventory-relative examples and claims must be rechecked.

The general definitions of operational identity, faithfulness, divergence, store composition, and finite decidability remain applicable when the revised interface remains finite, carrier-consistent, and decidable.

Three regimes in the imported inventory lie on no sibling axis. The audit of Section 4 decides faithfulness for all nine. Only the diagnosis of a failure uses $\mathrm{Sib}(\tau)$, and Section 4 states what is returned when $\mathrm{Sib}(\tau) = \varnothing$.

Remark 2.9 (Siblings Need Not Refine One Another). Neither declared relation on a sibling axis refines the other in general. A structural revision preserving rule content is non-breaking under RULE-C and breaking under RULE-S. A wording change altering content while preserving section structure is breaking under RULE-C and non-breaking under RULE-S. The two relations cross. The comparison of Section 4 is therefore carried out inside declared co-reference classes, where a containment can be asked for, rather than globally, where none may be assumed.

2.2 Preconditions on the Audit Artifact

The procedure consumes a declared regime assignment and a family-labeled history. It does not establish that either is correct.

Definition 2.10 (Audit Preconditions)**.** For an examined record domain R of a single carrier kind with declared regime τ:

(P1) *Regime determinacy.* Each record in R has exactly one effective declared regime, and that regime is τ.

(P2) *History well-formedness.* Every transformation in H acts on the carrier kind of R, so $\mathrm{Class}_\rho(f_i) \neq \mathsf{N/A}$ for every i and every regime ρ assigned to that kind.

(P3) *History-label validity.* Each family label f_i accurately names the transformation performed. The declared identity effect of that transformation is obtained by applying Class_τ to f_i. Any identity effect applied by the implementation is part of the identity-relevant treatment being audited and is not assumed to agree with the declaration.

Remark 2.11 (Preconditions Are Inputs). The preconditions are assumptions on the audit artifact. The declared partition is computed from the regime assignment and the recorded family labels. An indeterminate regime assignment, an unresolved endpoint, an unavailable family label, or a family label that does not accurately name the transformation performed makes the declared partition unreliable. A comparison against an unreliable declared partition carries no information about the surface. Failure to discharge one of those conditions yields INDETERMINATE, not PASS or FAIL.

A difference between the identity effect prescribed by $\mathrm{Class}_\tau(f_i)$ and the identity effect applied by the implementation is not a precondition failure. It is a candidate operational-identity divergence.

3 Surfaces and the Operational Identity Partition

Identity behavior may be implemented outside a formal regime declaration. A role may select an identifier policy. A workflow state may determine whether two revisions co-refer. A schema convention may change persistence handling. An application predicate may route records through different transformation rules.

The presence of such a mechanism is not a failure. A mechanism may serve functions unrelated to identity. The audit must first say what makes a mechanism identity-bearing.

3.1 The Surface Registry

Definition 3.1 (Mechanism Registry)**.** Let B be an identified implementation boundary and R the examined record domain. A *mechanism registry* D_B is a finite set of implementation artifacts within B.

Each $d \in D_B$ is an identifiable field, workflow state, schema convention, configuration value, application predicate, or policy input, together with a total computable observation map

$$d : R \longrightarrow V_d.$$

A missing or inapplicable value is represented by a distinguished value in V_d.

Each artifact is disclosed by the operator or independently discovered by the auditor.

Remark 3.2 (Why the Registry Is Necessary). Every function on the record domain induces a partition of it. Without the registry constraint, an auditor could take the identity function on records as a mechanism, which separates every distinct pair and therefore reports a divergence in any system that does anything at all with two co-referring records. The registry is what makes a surface an artifact of the implementation rather than an artifact of the audit. A candidate mechanism must be exhibitable: the auditor must be able to identify the implementation artifact that carries it.

Definition 3.3 (Joint Mechanism)**.** For a nonempty finite $S \subseteq D_B$, the *joint mechanism* d_S is

$$d_S(r) = \big\langle d(r) \;\big|\; d \in S \big\rangle,$$

under a fixed order on S. Write $\mathcal{D}_B$ for the set of joint mechanisms over nonempty finite subsets of D_B, which contains each $d \in D_B$ as the singleton case.

The auditor must exhibit the constituent mechanisms of a joint surface, so $\mathcal{D}_B$ is finite, every element of it is an exhibitable composite of registered artifacts, and the identity function on records is not among them unless D_B already separates every pair of records, in which case the implementation has itself disclosed a mechanism that does so.

3.2 Identity-Relevant Outcomes

Definition 3.4 (Identity-Relevant Outcome Types)**.** An outcome is *identity-relevant* when it is of one of the following types:

(a) *referent-identifier assignment:* which referent identifier a record is bound to;
(b) *referent resolution:* the referent identifier, referent key, or co-reference class to which the system resolves a record;
(c) *referent persistence:* whether a referent is continued, retired, or created;
(d) *transformation treatment:* a finite record-indexed encoding of the transformation family or identity effect applied to the record within the examined history;
(e) *applicability selection:* which identity rule or basis governs a record; or
(f) *operation admissibility:* whether an operation preserving or breaking identity is permitted.

Remark 3.5 (Scope of the Outcome Vocabulary). Definition 3.4 fixes the identity-relevant outcome vocabulary for the present audit. The enumeration leaves open whether other audit specifications may admit additional identity-relevant outcome types. A revision of the finite outcome vocabulary produces a revised audit specification, under which the identified uses, identity-treatment signatures, and operational partitions must be recomputed.

Remark 3.6 (Record Identifiers Are Not Identity-Relevant). Every record in a well-formed store carries a distinct record identifier by construction. A record identifier individuates the representation, rather than the referent. Two records co-referring under any regime therefore differ in record identifier, and admitting that difference as an identity-relevant outcome would produce a divergence witness in every system, including every correct one. The outcome types of Definition 3.4 are stated over referents throughout. The distinction between the two identifier layers is a precondition on the audit vocabulary, rather than a matter of exposition.

Remark 3.7 (Record-Indexed Encoding). The operational partition is induced by total record-indexed outcome functions.

A pairwise co-reference procedure may be represented, over finite R, by the vector of its decisions against the records of R under a fixed order. A transformation-level procedure may be represented by a fixed finite record-indexed encoding of the identity-relevant transformation outcomes incident to each record.

This encoding is part of the surface's identified use: it must be a total function of the record factoring through the mechanism value in the sense of Definition 3.8(c), and the operational partition induced by a transformation-treatment outcome is defined relative to it. Two auditors who fix different such encodings evaluate different uses, and the disclosure-relativity of Section 7 already ranges over that choice.

When no finite total record-indexed encoding is available, the procedure does not induce the operational partition defined here, and the verdict is INDETERMINATE.

3.3 Surfaces

Definition 3.8 (Audit Surface)**.** An *audit surface* over a record domain R of one carrier kind with declared regime τ consists of:

(a) a mechanism $d \in \mathcal{D}_B$ with a computable value function

$$d : R \longrightarrow V_d;$$

(b) a finite sequence of *identified* identity-relevant uses

$$U_d = \langle u_1, \ldots, u_m \rangle, \qquad u_j : R \longrightarrow O_j,$$

each of a type drawn from Definition 3.4, with finite, computable outcome for every $r \in R$. The sequence records the uses the audit has found, and Section 7 states the completeness claim under which it may be treated as all of them;

(c) *surface control:* each use factors through the mechanism value, so that for every j there exists

$$\tilde{u}_j : V_d \longrightarrow O_j \qquad \text{with} \qquad u_j = \tilde{u}_j \circ d \quad \text{on } R.$$

Remark 3.9 (Surface Granularity). Clause (c) is an admissibility condition on how the auditor identifies the mechanism. When the identity-relevant uses of a candidate mechanism do not factor through its value, the outcome is jointly controlled by that mechanism and others, and the auditor escalates to the joint mechanism d_S of Definition 3.3 for a subset $S \subseteq D_B$ that together determines the outcome. The escalation ranges over $\mathcal{D}_B$ and terminates, because D_B is finite. It cannot terminate in an arbitrary function of the record, which Remark 3.2 excludes. When no $S \subseteq D_B$

yields factorization, the audit returns INDETERMINATE: the disclosed registry does not contain the mechanisms that determine the outcome.

Definition 3.10 (Examined Surface Family)**.** An *examined surface family* A_B is a finite set of audit surfaces over $\mathcal{D}_B$, each supplied with its own identified-use sequence. The family is what the audit actually evaluates. It contains the singleton surfaces the auditor admitted and any joint surfaces of Definition 3.3 to which granularity escalation carried them.

The distinction between D_B, $\mathcal{D}_B$, and A_B does work. D_B is the disclosed inventory of implementation artifacts. $\mathcal{D}_B$ is its closure under joint value, which is what makes surface control attainable. A_B is the finite subfamily the audit evaluated, and it is the family over which store-level statements are made. An outcome jointly controlled by two registered fields and factoring through neither alone appears in A_B as a joint surface and nowhere else, so store-level conclusions must range over A_B rather than over D_B.

Definition 3.11 (Identity-Treatment Signature and Operational Partition)**.** For an audit surface d and record $r \in R$, define

$$\mathsf{IT}_d(r) = \langle u_1(r), \ldots, u_m(r) \rangle.$$

Write $r \approx_d s$ when $\mathsf{IT}_d(r) = \mathsf{IT}_d(s)$. The relation $\approx_d$ is an equivalence relation on R, and its quotient

$$\pi_d = R \,/\, \approx_d$$

is the *operational identity partition* induced by d.

The blocks of π_d are the sets of records to which the implementation gives the same identity-relevant treatment through the identified uses of d. It is derived from behavior, and it is available to an auditor whether or not the system's declaration mentions it. It is the rule of sameness the mechanism applies precisely to the extent that U_d is complete, which is a disclosure claim and not a computation. Definition 7.7 states that claim, and Definition 7.8 states the object that refutes it.

Lemma 3.12 (Treatment Is Determined by Mechanism Value)**.** *Let d be an audit surface and $r, s \in R$. If $d(r) = d(s)$ then $r \approx_d s$. Equivalently, $\mathsf{IT}_d(r) \neq \mathsf{IT}_d(s)$ implies $d(r) \neq d(s)$.*

Proof. By surface control, $u_j = \tilde{u}_j \circ d$ for each j. If $d(r) = d(s)$ then $u_j(r) = \tilde{u}_j(d(r)) = \tilde{u}_j(d(s)) = u_j(s)$ for every j, so the signatures agree. The second statement is the contrapositive. □

Lemma 3.12 says that π_d is coarser than the partition of R by mechanism value. A difference in identity treatment entails a difference in mechanism value, so an auditor who exhibits the former has already exhibited the latter. The witness of Section 4 therefore needs no separate clause on mechanism values, and reports them as evidence rather than requiring them as a condition.

The same fact disposes of the boundary case in Definition 3.3. A joint mechanism whose value separates every pair of records, that is, the case in which the registry is itself injective, still induces π_d through its uses, and those uses may merge records the value distinguishes. Value injectivity therefore splits no declared class on its own, and excluding the record-identity function (Remark 3.2) costs the auditor nothing on a system that has genuinely disclosed a pair-separating mechanism.

4 Comparing Declared and Operational Identity

Both π_τ and π_d are partitions of the same finite domain R. The audit is their comparison in the refinement lattice. Throughout, a partition π *refines* σ when every block of π is contained in a block of σ, equivalently when $\sim_\pi \subseteq \sim_\sigma$ as relations.

4.1 Faithfulness

Definition 4.1 (Faithful Surface)**.** An audit surface d is *faithful* to the declared regime τ on R when π_τ refines π_d, that is, when

$$r \sim_\tau s \implies r \approx_d s \qquad \text{for all } r, s \in R.$$

A faithful surface never splits a declared co-reference class. It may merge them.

Remark 4.2 (Why Faithfulness Is One-Directional)*.* The two directions of the refinement relation are not symmetric faults, and the asymmetry follows from what a single surface is.

A surface controls some identity-relevant outcomes. It does not carry the system's whole identity assignment. When a surface merges two records the declaration separates, it reports only that this mechanism does not itself distinguish those referents, and another mechanism may. A version field that is identical across two unrelated rules has told the audit nothing false.

When a surface splits a declared co-reference class, the system has already given the two records different referent identifiers, refused to resolve them together, or applied different persistence handling. No further mechanism can retract that treatment. The distinction has been made.

Conflation of declared-distinct referents is therefore a property of the store rather than of a surface, and it lies outside the scope of a single-surface audit. What a surface can be held to is that it introduces no distinction the declaration lacks.

Definition 4.3 (Divergence Witness)**.** A pair $(r, s) \in R \times R$ is a *divergence witness* for (d, τ) when

$$r \sim_\tau s \qquad \text{and} \qquad \mathsf{IT}_d(r) \neq \mathsf{IT}_d(s).$$

Proposition 4.4 (Divergence Refutes Faithfulness)**.** *A surface d is faithful to τ on R if and only if no divergence witness for (d, τ) exists in $R \times R$. Every divergence witness satisfies $d(r) \neq d(s)$.*

Proof. Definition 4.1 is a universally quantified implication whose negation at a pair is exactly Definition 4.3. The second statement is Lemma 3.12. □

A divergence witness is an audit finding on its own. It shows that identity treatment in the deployed system depends on a distinction the declared regime does not make. It is available for every regime in the inventory, including the three that lie on no sibling axis. It does not yet say which distinction.

4.2 Faithfulness Composes

A single surface carries part of the system's identity treatment. The examined identity treatment is carried by A_B.

Definition 4.5 (Store Operational Partition)**.** For an examined surface family A_B on R, define

$$\approx_B = \bigcap_{d \in A_B} \approx_d, \qquad \pi_B = R \,/\, \approx_B .$$

Two records are operationally the same in the store when every examined surface, singleton or joint, gives them the same identity-relevant treatment.

Proposition 4.6 (Store Faithfulness Is Surface Faithfulness)**.** *The declared partition π_τ refines π_B if and only if every $d \in A_B$ is faithful to τ on R. A divergence witness for any surface in A_B is a divergence witness for the store.*

Proof. $\approx_B$ is an intersection over A_B, so $\sim_\tau \subseteq \approx_B$ holds exactly when $\sim_\tau \subseteq \approx_d$ for every $d \in A_B$. A pair witnessing the failure of one conjunct witnesses the failure of the whole. □

Faithfulness therefore composes. A divergence on any surface refutes store faithfulness, while the store passes only when every surface in A_B is faithful. The audit may proceed one surface at a time while retaining those quantified conclusions.

The quantification over A_B rather than D_B is not cosmetic. An outcome jointly controlled by two registered fields, factoring through neither alone, is carried by a joint surface. That surface may split a declared co-reference class while each constituent field, taken alone, splits nothing. Intersecting only over the singleton mechanisms of D_B would omit the surface that carries the fault, and Proposition 4.6 would then be false.

Under the completeness claims of Section 7, π_B is the rule of sameness the system implements. Proposition 4.6 licenses a store-level failure from any failing surface and a store-level pass only from faithfulness of every surface in A_B.

4.3 The Lattice Comparison

Diagnosis restricts both relations to the declared co-reference classes.

Definition 4.7 (Restricted Relations)**.** Let $\tau' = \mathrm{Sib}(\tau) \neq \varnothing$. Define the equivalence relations

$$\Delta_d^{\mathrm{op}} = \approx_d \ \cap \ \sim_\tau, \qquad \Delta^{\mathrm{sib}} = \sim_{\tau'} \ \cap \ \sim_\tau \ .$$

Both are equivalence relations contained in $\sim_\tau$. Δ_d^{op} records which pairs the declaration merges and the surface also merges. Δ^{sib} records which pairs the declaration merges and the sibling basis also merges. Restricting to $\sim_\tau$ is what makes the comparison well posed, given Remark 2.9.

Faithfulness is now the statement $\Delta_d^{\mathrm{op}} = \sim_\tau$. When it fails, Δ_d^{op} is a proper subrelation of $\sim_\tau$. If Δ^{sib} is also a proper subrelation of $\sim_\tau$, the two restricted relations fall into one of four cases. If $\Delta^{\mathrm{sib}} = \sim_\tau$, the divergence is unpositioned.

Definition 4.8 (Divergence Classification)**.** Suppose d is not faithful to τ on R, that $\mathrm{Sib}(\tau) = \tau' \neq \varnothing$, and that $\Delta^{\mathrm{sib}} \neq \sim_\tau$, so the sibling basis also splits some declared class. Exactly one of the following holds:

(a) $\Delta_d^{\mathrm{op}} = \Delta^{\mathrm{sib}}$: *sibling-aligned divergence*. Inside the declared classes, the surface splits exactly as the sibling does.

(b) $\Delta_d^{\mathrm{op}} \subsetneq \Delta^{\mathrm{sib}}$: *sub-sibling divergence*. Inside the declared classes, the surface splits strictly more finely than the sibling.

(c) $\Delta^{\mathrm{sib}} \subsetneq \Delta_d^{\mathrm{op}}$: *super-sibling divergence*. Inside the declared classes, the surface makes some but not all of the sibling's distinctions.

(d) Δ_d^{op} and Δ^{sib} are incomparable: *sibling-incomparable divergence*. The surface makes distinctions the sibling does not and omits distinctions the sibling makes.

When $\mathrm{Sib}(\tau) = \varnothing$, or when $\Delta^{\mathrm{sib}} = \sim_\tau$, the divergence is *unpositioned*: the surface splits a declared class, and the imported inventory supplies no sibling distinction on R against which the split can be located.

Under the hypotheses of Definition 4.8, the four cases are exhaustive and mutually exclusive, being the four possible relative positions of two elements of a partially ordered set under the containment order.

Remark 4.9 (What the Classification Does Not Say). Definition 4.8 compares Δ_d^{op} and Δ^{sib}, both of which are intersected with $\sim_\tau$. The comparison is therefore local to the declared co-reference classes, and it is silent about pairs the declaration separates. Sibling-aligned divergence establishes that the surface splits declared classes as the sibling would. It does not establish that $\approx_d = \sim_{\tau'}$ on R, because the surface may treat a τ-separated pair in a way the sibling does not. The names of the four cases are relational for that reason, and none of them asserts that the surface implements a named basis.

Definition 4.10 (Regime Substitution and Hidden Identity Regime)**.** A surface d exhibits *regime substitution* on R when $\mathrm{Sib}(\tau) = \tau' \neq \varnothing$, the regimes differ on R, and the operational partition equals the sibling partition,

$$\pi_d = \pi_{\tau'}, \qquad \text{equivalently} \qquad \approx_d = \sim_{\tau'} \text{ on } R.$$

A surface exhibiting regime substitution implements a *hidden identity regime*: a consistently applied, undeclared rule of sameness that the imported inventory already names.

Example 4.11 (A Forked Continuation on the Locus/Object Axis)**.** Let r_0 record a monitoring station at site ℓ using sensor o_0. A branch/fork creates two continuation records r_1 and r_2 for monitoring at the same site ℓ, using sensor objects o_1 and o_2, with o_0, o_1, and o_2 carrying distinct serials. Let

$$R = \{r_0, r_1, r_2\}$$

and

$$H = \big\langle (\mathsf{BF}, r_0, r_1), (\mathsf{BF}, r_0, r_2) \big\rangle.$$

In the imported referential-regime interface,

$$\mathrm{Class}_{\mathrm{LOC}}(\mathsf{BF}) = \mathsf{PRS} \qquad \text{and} \qquad \mathrm{Class}_{\mathrm{OBJ}}(\mathsf{BF}) = \mathsf{BRK}.$$

Both fork edges are therefore non-breaking under LOC, giving

$$\pi_{\mathrm{LOC}} = \big\{\{r_0, r_1, r_2\}\big\}.$$

Both are breaking under OBJ, giving

$$\pi_{\mathrm{OBJ}} = \big\{\{r_0\}, \{r_1\}, \{r_2\}\big\}.$$

Suppose the declared regime is LOC. Let d be the registered sensor-serial field, with a referent-identifier use that assigns a distinct station identifier to each sensor serial, so the station's referent identifier follows the sensor object. Then

$$\pi_d = \pi_{\mathrm{OBJ}}.$$

The pair (r_0, r_1) is a divergence witness, because the declaration merges it and the surface separates it. The divergence is sibling-aligned, and the equality $\pi_d = \pi_{\mathrm{OBJ}}$ establishes regime substitution.

Regime substitution is a global condition on R, and it is the condition an earlier formulation of this result treated as the whole of it. The next subsection separates it from sibling alignment and from a single witness pair.

4.4 Neither a Witness nor an Alignment Diagnoses

Proposition 4.12 (Substitution Implies Sibling Alignment and a Witness)**.** *Suppose d exhibits regime substitution on R and that*

$$\Delta^{\mathrm{sib}} \neq \sim_\tau, \qquad \textit{equivalently} \qquad \sim_\tau \not\subseteq \sim_{\tau'} .$$

Then d is not faithful to τ on R, it exhibits sibling-aligned divergence, and any pair (r, s) with $r \sim_\tau s$ and $r \not\sim_{\tau'} s$ is a divergence witness for (d, τ) that separates under the sibling.

Proof. Substitution gives $\approx_d \; = \; \sim_{\tau'}$, so intersecting both sides with $\sim_\tau$ gives $\Delta_d^{\mathrm{op}} = \Delta^{\mathrm{sib}}$. The hypothesis $\Delta^{\mathrm{sib}} \neq \sim_\tau$ supplies a pair with $r \sim_\tau s$ and $r \not\sim_{\tau'} s$, which satisfies $\mathsf{IT}_d(r) \neq \mathsf{IT}_d(s)$ because $\approx_d \; = \; \sim_{\tau'}$. That pair is a divergence witness, so faithfulness fails, and $\Delta_d^{\mathrm{op}} = \Delta^{\mathrm{sib}}$ with $\Delta^{\mathrm{sib}} \neq \sim_\tau$ is sibling-aligned divergence. □

Remark 4.13 (A Hidden Sibling Regime Need Not Be a Splitting Fault)*.* The hypothesis of Proposition 4.12 is not implied by the requirement that the regimes differ on R, and dropping it makes the proposition false.

Suppose the examined history contains only a content amendment preserving section structure, which is BRK under RULE-C and PRS under RULE-S. Then $\sim_{\text{RULE-C}}$ separates the two records and $\sim_{\text{RULE-S}}$ merges them, so the regimes differ on R while $\sim_{\text{RULE-C}} \subseteq \sim_{\text{RULE-S}}$. A surface with $\pi_d = \pi_{\text{RULE-S}}$ exhibits regime substitution: it globally implements the undeclared sibling basis. It nevertheless splits no declared co-reference class, so by Remark 4.2 it is faithful and the audit returns PASS.

The surface is running an identity regime different from the declaration, and it is running it by merging referents the declaration separates, which is the direction this audit does not police. The observation is a limit on the audit rather than a defect in the definitions. A splitting fault is refutable by one pair because no later mechanism can retract a distinction already made. A merging fault is not, because another mechanism in A_B may carry the distinction, and Definition 4.5 is where a store-level merging analysis would have to begin.

Proposition 4.14 (Sibling Alignment Does Not Imply Substitution)**.** *There is a carrier kind, a declared regime τ with sibling τ', a family-labeled history over the*

imported transformation basis, and a surface d exhibiting sibling-aligned divergence and not exhibiting regime substitution.

Proof. Take rules, $\tau = \text{RULE-C}$, $\tau' = \text{RULE-S}$, $R = \{r_0, r_1, r_2, r_3\}$, and

$$H = \big\langle (\mathsf{refine\text{-}structure},\, r_0,\, r_1),\ (\mathsf{revise\text{-}wording},\, r_0,\, r_2),\ (\mathsf{amend\text{-}content},\, r_0,\, r_3) \big\rangle.$$

Structural refinement preserves content and changes structure, so it is PRS under RULE-C and BRK under RULE-S. Content-preserving rewording changes neither, so it is PRS under both. A content amendment preserving section structure changes content and leaves structure fixed, so it is BRK under RULE-C and PRS under RULE-S by Remark 2.9. Therefore

$$\pi_{\text{RULE-C}} = \big\{\{r_0, r_1, r_2\}, \{r_3\}\big\}, \qquad \pi_{\text{RULE-S}} = \big\{\{r_0, r_2, r_3\}, \{r_1\}\big\},$$

and

$$\Delta^{\text{sib}} = \operatorname{diag}(R) \cup \big\{(r_0, r_2), (r_2, r_0)\big\}.$$

Let d be a registered mechanism with

$$d(r_0) = d(r_2) = 0, \qquad d(r_1) = d(r_3) = 1,$$

and a single use of referent-identifier assignment given by $\tilde{u}(0) = \mathsf{a}$ and $\tilde{u}(1) = \mathsf{b}$ for distinct referent identifiers $\mathsf{a} \neq \mathsf{b}$. Surface control holds by construction, $\mathsf{IT}_d = \tilde{u} \circ d$, and

$$\pi_d = \big\{\{r_0, r_2\}, \{r_1, r_3\}\big\}.$$

Then

$$\Delta_d^{\text{op}} \;= \approx_d \;\cap\; \sim_{\text{RULE-C}} \;=\; \Delta^{\text{sib}},$$

so d exhibits sibling-aligned divergence. But $\pi_d \neq \pi_{\text{RULE-S}}$, since d separates r_0 from r_3 while RULE-S merges them, and d merges r_1 with r_3 while RULE-S separates them. The disagreement lies entirely on pairs that RULE-C separates, which Δ_d^{op} and Δ^{sib} do not see. □

Proposition 4.15 (A Sibling-Separating Witness Does Not Establish Alignment)**.** *There is a carrier kind, a declared regime τ with sibling τ', a family-labeled history over the imported transformation basis, and a surface d such that a divergence witness separating under τ' exists, and d exhibits sub-sibling divergence rather than sibling-aligned divergence.*

Proof. Take the carrier kind of rules, declared regime τ = RULE-C, sibling τ' = RULE-S, and the record domain $R = \{r_0, r_1, r_2\}$ with history

$$H = \big\langle (\mathsf{refine\text{-}structure},\, r_0,\, r_1),\ (\mathsf{revise\text{-}wording},\, r_0,\, r_2) \big\rangle.$$

Structural refinement preserves rule content and changes section structure, so it is PRS under RULE-C and BRK under RULE-S. Content-preserving rewording changes neither content nor structure, so it is PRS under both. Hence

$$E_{\text{RULE-C}} = \{(r_0, r_1), (r_0, r_2)\}, \qquad E_{\text{RULE-S}} = \{(r_0, r_2)\},$$

giving one declared block $\{r_0, r_1, r_2\}$ and sibling blocks $\{r_0, r_2\}$ and $\{r_1\}$.

Let d be a registered field holding a version counter that increments whenever the stored rule text changes, so d takes three distinct values on R, and let its single use be referent-identifier assignment, returning a distinct referent identifier for each value. Surface control holds, and $\approx_d$ is the identity relation on R.

The pair (r_0, r_1) satisfies $r_0 \sim_{\text{RULE-C}} r_1$ and $\mathsf{IT}_d(r_0) \neq \mathsf{IT}_d(r_1)$, so it is a divergence witness, and $r_0 \not\sim_{\text{RULE-S}} r_1$, so it separates under the sibling.

The pair (r_0, r_2) satisfies $r_0 \sim_{\text{RULE-C}} r_2$ and $r_0 \sim_{\text{RULE-S}} r_2$ while $\mathsf{IT}_d(r_0) \neq \mathsf{IT}_d(r_2)$, so $(r_0, r_2) \in \Delta^{\text{sib}} \setminus \Delta^{\text{op}}_d$. Hence $\Delta^{\text{op}}_d \subsetneq \Delta^{\text{sib}}$, which is sub-sibling divergence. □

The construction uses only a version counter that increments on any change to stored text. On the examined records it splits declared classes strictly more finely than either imported basis. The pair (r_0, r_1) is a valid divergence witness, but it does not establish the classification it appears to invite. Section 6 shows why the classification matters: the repair suggested by sibling alignment would leave this fault in place.

Remark 4.16 (Non-vacuity of the Classification). Cases (a) and (b) of Definition 4.8 are realized by Example 4.11 and Proposition 4.15. Cases (c) and (d) also occur under the imported interface, so the four-way classification is not partly empty.

Super-sibling (c). Take τ = RULE-C, τ' = RULE-S, $R = \{r_0, r_1, r_2\}$, and

$$H = \big\langle (\mathsf{refine\text{-}structure},\, r_0,\, r_1),\ (\mathsf{refine\text{-}structure},\, r_0,\, r_2) \big\rangle.$$

Both edges are PRS under RULE-C and BRK under RULE-S, so

$$\pi_{\text{RULE-C}} = \big\{\{r_0, r_1, r_2\}\big\}, \qquad \pi_{\text{RULE-S}} = \big\{\{r_0\}, \{r_1\}, \{r_2\}\big\},$$

giving $\Delta^{\text{sib}} = \text{diag}(R)$ on the declared class. Let d be a registered field with $d(r_0) = d(r_1) = 0$ and $d(r_2) = 1$, and a single use of referent-identifier assignment given by $\tilde{u}(0) = \mathsf{a}$ and $\tilde{u}(1) = \mathsf{b}$ for distinct referent identifiers $\mathsf{a} \neq \mathsf{b}$. Then $\approx_d$ has blocks $\{r_0, r_1\}$ and $\{r_2\}$, so

$$\Delta_d^{\text{op}} = \text{diag}(R) \cup \big\{(r_0, r_1), (r_1, r_0)\big\} \qquad \text{and} \qquad \Delta^{\text{sib}} \subsetneq \Delta_d^{\text{op}} \subsetneq \sim_{\text{RULE-C}}.$$

The surface makes one of the two structural distinctions the sibling makes and omits the other.

Sibling-incomparable (d). Take the same τ, τ', with $R = \{r_0, r_1, r_2, r_3\}$ and

$$H = \big\langle(\textsf{revise-wording},\, r_0,\, r_1),\ (\textsf{refine-structure},\, r_0,\, r_2),\ (\textsf{revise-wording},\, r_2,\, r_3)\big\rangle.$$

All three edges are PRS under RULE-C, so $\pi_{\text{RULE-C}} = \big\{\{r_0, r_1, r_2, r_3\}\big\}$. Under RULE-S the two rewordings are PRS and the refinement is BRK, so $\pi_{\text{RULE-S}} = \big\{\{r_0, r_1\}, \{r_2, r_3\}\big\}$, and Δ^{sib} merges (r_0, r_1) and (r_2, r_3). Let d be a registered field with $d(r_0) = d(r_2) = 0$ and $d(r_1) = d(r_3) = 1$, and a single use of referent-identifier assignment given by $\tilde{u}(0) = \mathsf{a}$ and $\tilde{u}(1) = \mathsf{b}$ for distinct referent identifiers $\mathsf{a} \neq \mathsf{b}$. So $\approx_d$ has blocks $\{r_0, r_2\}$ and $\{r_1, r_3\}$. Then

$$(r_0, r_1) \in \Delta^{\text{sib}} \setminus \Delta_d^{\text{op}} \qquad \text{and} \qquad (r_0, r_2) \in \Delta_d^{\text{op}} \setminus \Delta^{\text{sib}},$$

so Δ_d^{op} and Δ^{sib} are incomparable. In both constructions surface control holds through the single use, and d splits the declared class, so the audit returns FAIL.

Propositions 4.14 and 4.15 bound the diagnostic strength of the audit from both sides. A witness does not establish a classification, and a classification does not establish a basis. The audit establishes that the declaration and the implementation disagree and, when the sibling comparison is informative, where their disagreement lies in the lattice.

Propositions 4.14 and 4.15 bound the diagnostic strength of the audit from both sides. A witness does not establish a classification and a classification does not establish a basis. The audit establishes that the declaration and the implementation

disagree and, when the sibling comparison is informative, where their disagreement lies in the lattice.

5 Decidability, Verdict, and Non-monotonicity

Assumption 5.1 (Finite Audit Artifact)**.** For each examined surface d:

(a) the preconditions of Definition 2.10 are discharged;
(b) R and the relevant transformation history H are finite;
(c) every history endpoint resolves to a record in R;
(d) each family f_i is known and $\mathrm{Class}_\rho(f_i)$ is decidable for each regime assigned to the carrier kind;
(e) $d \in A_B$, so $d \in \mathcal{D}_B$ and $d(r)$ and $\mathsf{IT}_d(r)$ are computable for every $r \in R$;
(f) surface control is established for d over $\mathcal{D}_B$.

Store-level statements range over the examined family A_B.

Theorem 5.2 (Finite Decidability)**.** *Under Assumption 5.1, the declared partition π_τ, the operational partition π_d, faithfulness of d to τ on R, existence of a divergence witness, and the divergence classification of Definition 4.8 are all computable.*

Let

$$n = |R|, \qquad h = |H|, \qquad m = |U_d|.$$

Constructing the declared and sibling partitions requires at most $2h$ calls to the imported classification functions and $O((n+h)\alpha(n))$ union-find time, where α is the inverse-Ackermann function. Constructing the operational partition requires mn identified-use evaluations and grouping the resulting signatures. Once the partitions are built, faithfulness, witness extraction, and divergence classification take at most $O(n^2)$ time by pair enumeration and expected $O(n)$ time using block labels and hashing.

Proof. The checker computes E_τ and, when $\mathrm{Sib}(\tau) \neq \varnothing$, $E_{\tau'}$, from the finite history, using decidability of the classification functions. It forms $\sim_\tau$ and $\sim_{\tau'}$ as equivalence closures by union-find over the finite edge sets, giving π_τ and the sibling partition in $O((n+h)\alpha(n))$ union-find time.

It computes $\mathsf{IT}_d(r)$ for every $r \in R$ and groups records by signature, giving π_d. This requires mn identified-use evaluations. Apart from the costs of evaluating and representing individual outcomes, constructing and hashing the signatures takes expected $O(mn)$ time.

Faithfulness is the containment $\sim_\tau \subseteq \approx_d$. It is decidable by enumerating the pairs in each block of π_τ and comparing their signatures. Any pair at which the containment fails is a divergence witness.

Suppose faithfulness fails. When $\mathrm{Sib}(\tau) = \varnothing$, the divergence is computably classified as unpositioned. When a sibling exists, the checker computes Δ^{sib}; if $\Delta^{\mathrm{sib}} = \sim_\tau$, the divergence is again classified as unpositioned. Otherwise, the checker computes Δ_d^{op} and performs the containment tests

$$\Delta_d^{\mathrm{op}} \subseteq \Delta^{\mathrm{sib}} \qquad \text{and} \qquad \Delta^{\mathrm{sib}} \subseteq \Delta_d^{\mathrm{op}}.$$

The four possible outcomes are exactly the four sibling-relative cases of Definition 4.8.

Pair enumeration gives a deterministic $O(n^2)$ upper bound for this comparison stage. For the near-linear implementation, assign each record its block labels in π_τ, π_d, and, when present, $\pi_{\tau'}$. Faithfulness holds exactly when every declared block has a single operational label; retaining one representative record for each declared block produces a witness when a second label is encountered. The containment $\Delta_d^{\mathrm{op}} \subseteq \Delta^{\mathrm{sib}}$ holds exactly when every block identified by its declared and operational labels has a single sibling label. The reverse containment holds exactly when every block identified by its declared and sibling labels has a single operational label. Hash tables perform these scans in expected $O(n)$ time. □

5.1 Three-Valued Verdict

Definition 5.3 (Audit Verdict)**.** The audit of a surface d against declared regime τ on R returns:

- PASS when the preconditions and surface control are discharged and d is faithful to τ on R;
- FAIL when they are discharged and a divergence witness exists, reported with the classification of Definition 4.8 and with a witness pair;
- INDETERMINATE when a precondition is not discharged, surface control cannot

be established over $\mathcal{D}_B$, a family label is unavailable or unverified, or an identity-relevant outcome cannot be computed for some record in R.

Remark 5.4 (Indeterminate Is Not Conformance). An INDETERMINATE verdict reports that the audit could not decide. It has the standing of an unexamined surface and does not support a claim that the surface is faithful. Privacy, security, trade-secret, and access constraints frequently produce INDETERMINATE rather than PASS, and reporting the two under one heading would let an unexaminable system present as a conformant one.

5.2 Passing Is Not Monotone

The declared partition is built by transitive closure. That fact has a consequence for what a passing verdict can be indexed to.

Proposition 5.5 (Non-monotonicity of PASS under History Extension)**.** *There exist a fixed record domain R, histories $H \preceq H^*$ over R, and a surface d such that d is faithful to τ on (R, H) and a divergence witness for (d, τ) exists on (R, H^*). The record domain, the registry, the surface, the records of the witness, and their identity treatment are all unchanged.*

Proof. Let $R = \{r, s, t\}$ and let $H = \langle\rangle$ be the empty history, so $E_\tau = \varnothing$, the relation $\sim_\tau$ is the identity relation on R, and π_τ consists of three singleton blocks. Let d be a registered mechanism taking three distinct values on R, with a referent-identifier use returning a distinct identifier for each, so IT_d separates all three records. No pair (x, y) with $x \neq y$ satisfies $x \sim_\tau y$, so no divergence witness exists and d is faithful on (R, H).

Extend the history alone to

$$H^* = \big\langle (f,\, r,\, t),\ (g,\, t,\, s) \big\rangle,$$

where f and g are families non-breaking under τ. The record domain is still R, and both endpoints of both transformations lie in it. Now (r, t) and (t, s) lie in E_τ, and the equivalence closure gives $r \sim_\tau s$, so π_τ has collapsed to the single block $\{r, s, t\}$. By surface control IT_d is a function of d, so the signatures are unchanged and $\mathsf{IT}_d(r) \neq \mathsf{IT}_d(s)$ still holds. The pair (r, s) is now a divergence witness on (R, H^*). □

Corollary 5.6 (Indexing of a Passing Verdict)**.** *For a fixed examined surface and identified-use sequence, a* PASS *verdict is indexed to*

$$(\mathfrak{R}, R, H)$$

and not to R alone. The audit must be rerun when the transformation history is extended or the imported referential-regime interface is revised, even when the record domain, the registry, and the surface are unchanged.

This index records the history and interface dependence only. A pass verdict is additionally relative to the disclosed registry D_B, the evaluated surface family A_B, and the identified uses U_d of each surface; Section 7 states those dependences and the witnesses that refute them.

Non-monotonicity distinguishes this audit from a test whose passing result is stable under the arrival of new data. With R and the operational signatures fixed, the declared partition can only coarsen as history accumulates. That coarsening can create divergence witnesses but cannot remove an existing one. An operator who obtains a passing verdict holds a statement about a history, not a certificate about a store.

6 Worked Legal-Alignment Example

Consider a legal-alignment system recording a reporting rule used to evaluate a regulatory filing submitted by an AI-enabled agent. Advanced agents may act within legal and institutional processes, and Kolt describes a trajectory in which they function as subjects, consumers, producers, and enforcers of law (Kolt 2026). Empirical survey of deployed agentic systems reports uneven documentation and limited disclosure across the ecosystem (Staufer et al. 2026), which complicates later inspection of which rules, filings, and authorities an accountability claim concerns.

The legal conclusion may remain contested. The identity of the rule must remain stable enough for reviewers to examine the same authority while disputing whether the filing complied with it.

6.1 Records and Declared Partition

Let r_0 be the original rule record. Let r_1 be a structural refinement: it adds subsections and renumbers sections while preserving the rule content. Let r_2 be an editorial revision of r_0 that rewords a sentence for clarity while preserving both content and section structure.

The system declares content-fixed rule identity, RULE-C. Its imported sibling is structure-fixed rule identity, RULE-S. By the classification used in Proposition 4.15, the declared partition is the single block

$$\pi_{\text{RULE-C}} = \big\{\{r_0, r_1, r_2\}\big\},$$

and the sibling partition is

$$\pi_{\text{RULE-S}} = \big\{\{r_0, r_2\}, \{r_1\}\big\}.$$

All three records refer to one rule under the declaration. The sibling basis would separate the structurally refined expression.

6.2 The Surface and Its Operational Partition

The registry D_B contains the field

$$d = \texttt{ruleTextVersion},$$

which increments whenever the stored rule text changes. All three records carry distinct values.

A value difference alone is harmless. The field may identify a displayed expression, support provenance, or route a reviewer to the relevant text. It becomes a surface only when identity-relevant uses factor through its value.

Suppose the application binds the referent identifier of a rule to the value of `ruleTextVersion`, so that when the field changes the application issues a new rule identifier, refuses co-reference with the previous expression, and resolves later citations to the new rule. Those uses are functions of the field value, so surface

control holds. The operational identity partition is

$$\pi_d = \big\{\{r_0\}, \{r_1\}, \{r_2\}\big\}.$$

6.3 Comparison

The declared partition does not refine the operational one: the block $\{r_0, r_1, r_2\}$ is split. The surface is not faithful, and the audit returns FAIL. The pair (r_0, r_1) is a divergence witness, and it separates under the sibling.

Restricting to the declared class gives

$$\Delta_d^{\mathrm{op}} = \big\{(r_0, r_0), (r_1, r_1), (r_2, r_2)\big\}, \qquad \Delta^{\mathrm{sib}} = \Delta_d^{\mathrm{op}} \cup \big\{(r_0, r_2), (r_2, r_0)\big\},$$

so $\Delta_d^{\mathrm{op}} \subsetneq \Delta^{\mathrm{sib}}$. The classification is sub-sibling divergence.

The operational and sibling partitions differ: $\pi_d = \big\{\{r_0\}, \{r_1\}, \{r_2\}\big\}$ separates r_0 from r_2, which $\pi_{\text{RULE-S}} = \big\{\{r_0, r_2\}, \{r_1\}\big\}$ merges.

Therefore $\approx_d \neq \sim_{\text{RULE-S}}$ on R, so the surface does not exhibit regime substitution (Definition 4.10), and the audit reports no hidden identity regime.

6.4 Why the Classification Changes the Repair

Reported as sibling alignment, the finding invites one repair: declare structure-fixed rule identity and document the basis. Applied here, that repair leaves the fault in place. The application would continue to break co-reference on r_2, an editorial revision that structure-fixed identity preserves. The operator would have declared RULE-S and still not be running it.

Reported as sub-sibling divergence, the finding says what is true. On the examined records the operational partition is total, so any change to the stored text yields a distinct rule referent. The split is strictly finer than either imported basis, and the imported inventory contains no basis that produces it. The audit does not name the basis the application carries, and the natural reading of π_d on R is text-fixed rule identity, which the operator can confirm from the field's update rule.

Three repairs follow, and the audit chooses among none of them. The first preserves

the declared regime: the version field remains available for display and provenance, the referent identifier is unbound from it, and every content-preserving expression resolves to the same rule. The surface values continue to differ, the operational partition coarsens to the declared one, and the surface becomes faithful. The second declares structure-fixed identity and repairs the implementation to match it, which requires the field to stop firing on editorial revision. The third adopts text-fixed identity as the intended basis, which requires an argument that it is stability-critical and a statement of its transformation classification, and which places the declared basis outside the imported inventory.

The audit establishes that the declaration and the examined implementation behavior diverge. It also establishes which distinctions the implementation carries on the examined records and where those distinctions lie relative to the declared and sibling partitions. It does not name the identity basis responsible for them.

7 Discovery and the Disclosure Boundary

The procedure evaluates the surfaces in the registry. It does not discover every mechanism inside arbitrary source code, workflow configuration, remote services, operational practice, or undocumented organizational convention.

Candidate mechanisms may be found through source-code analysis, schema and configuration inspection, workflow-rule extraction, runtime tracing, differential testing, policy-engine inspection, legal discovery, or post-incident review. Those methods answer *where might identity treatment be controlled?* The procedure of Section 5 answers *which rule of sameness does this mechanism carry, and is it the declared one?*

Definition 7.1 (Registry-Completeness Claim)**.** A *registry-completeness claim* for (B, D_B) asserts that every mechanism within B that affects identity-relevant treatment is represented in D_B.

The procedure can evaluate every surface in the examined family A_B. It cannot derive the truth of the completeness claim from D_B itself.

Definition 7.2 (Omitted-Mechanism Witness)**.** An *omitted-mechanism witness* for (B, D_B) consists of:

(a) a mechanism d^* operating within B with $d^* \notin D_B$;
(b) the declared regime τ for the carrier kind of its record domain;
(c) records r, s within the examined system or preserved audit history;
(d) evidence that the identity-relevant uses of d^* factor through its value;
(e) evidence that $r \sim_\tau s$; and
(f) evidence that $\mathsf{IT}_{d^*}(r) \neq \mathsf{IT}_{d^*}(s)$.

The omitted-mechanism witness requires divergence and does not require a sibling comparison. An undisclosed mechanism that splits a declared co-reference class is an identity-relevant mechanism absent from the disclosure, whatever basis it turns out to carry.

Proposition 7.3 (Later Witness Refutes Completeness)**.** *An omitted-mechanism witness within B refutes the registry-completeness claim for (B, D_B).*

Proof. The witness exhibits a mechanism within B and absent from D_B. Clauses (d) through (f) establish that it controls a difference in identity-relevant treatment between records the declaration merges, so it affects identity-relevant treatment. The claim that D_B contains every such mechanism within B is false. □

7.1 Family Completeness

Registry completeness bounds which implementation artifacts the audit has seen. It does not bound which *surfaces* over those artifacts the audit evaluated.

The gap is created by the closure $\mathcal{D}_B$. An identity-relevant use may factor through the joint mechanism $d_{\{1,2\}}$ and through neither d_1 nor d_2 alone. Suppose $D_B = \{d_1, d_2\}$, suppose the registry-completeness claim is true, and suppose A_B contains only the two singleton surfaces. The use-completeness claim for each singleton can be true, because the jointly controlled use is not a use of either singleton in the factorization sense of Definition 3.8(c). Each examined surface can pass. The joint surface can nevertheless split a declared co-reference class, and π_B does not see it. A simpler instance is a registered artifact the auditor never admits into A_B at all.

Definition 7.4 (Family-Completeness Claim)**.** A *family-completeness claim* for (B, D_B, A_B) asserts that, for every identity-relevant use within B whose controlling

mechanisms are represented in D_B, A_B contains at least one audit surface through whose mechanism that use factors.

Definition 7.5 (Omitted-Surface Witness)**.** An *omitted-surface witness* for a triple (B, D_B, A_B) consists of:

(a) an identity-relevant use u^* operating within B;

(b) a nonempty set $S \subseteq D_B$, the associated joint mechanism $d_S \in \mathcal{D}_B$, and a map $\tilde{u}^*$ such that

$$u^* = \tilde{u}^* \circ d_S \quad \text{on } R;$$

(c) for every examined surface $(d, U_d) \in A_B$, there is no map v such that

$$u^* = v \circ d \quad \text{on } R;$$

(d) records $r, s \in R$ with $r \sim_\tau s$; and

(e) evidence that $u^*(r) \neq u^*(s)$.

Proposition 7.6 (Omitted Surface Refutes Family Completeness)**.** *An omitted-surface witness refutes the family-completeness claim for its triple* (B, D_B, A_B)*. Suppose the audit returned* PASS *for every surface in* A_B*, and let*

$$A_B^* = A_B \cup \big\{(d_S, \langle u^* \rangle)\big\}.$$

Then the audit returns FAIL *for* d_S*, and by Proposition 4.6 the store verdict over* A_B^* *is* FAIL*.*

Proof. Clause (b) establishes that the controlling artifacts of u^* are represented in D_B. Clause (c) establishes that u^* factors through no examined surface mechanism. The family-completeness claim is therefore false.

Clause (b) also establishes surface control for $(d_S, \langle u^* \rangle)$, so it is an admissible audit surface. Clauses (d) and (e) make (r, s) a divergence witness for that surface. □

The omitted-surface witness is not an omitted-mechanism witness. Its constituent artifacts are already registered, so the registry-completeness claim survives it intact. What it refutes is the coverage of the evaluation, and the repair is to evaluate the joint surface, not to disclose a new artifact.

7.2 Use Completeness

The two prior boundaries bound which mechanisms the audit has seen and which surfaces over them it evaluated. A third bounds which *uses* of an evaluated surface it has seen.

The operational partition π_d is computed from U_d, the identified identity-relevant uses of d. Nothing in the computation requires U_d to contain every identity-relevant use of d within B. An operator who discloses a harmless use of a field and omits the use that binds it to referent-identifier assignment supplies a coarser π_d, and the audit may return PASS on a surface that in fact diverges.

Definition 7.7 (Use-Completeness Claim)**.** A *use-completeness claim* for (d, B, U_d) asserts that every identity-relevant use of d within B, in the sense of Definition 3.4, appears in U_d.

Definition 7.8 (Omitted-Use Witness)**.** An *omitted-use witness* for (d, B, U_d) consists of:

(a) a use u^* operating within B with $u^* \notin U_d$, of an outcome type in Definition 3.4;

(b) *surface control:* a map $\tilde{u}^*$ with

$$u^* = \tilde{u}^* \circ d \quad \text{on } R,$$

so the omitted use is a use *of* d in the sense of Definition 3.8(c);

(c) records $r, s \in R$ with $r \sim_\tau s$; and

(d) evidence that $u^*(r) \neq u^*(s)$.

Clause (b) is what keeps the expanded surface admissible. An identity-relevant outcome that does not factor through d is not an omitted use of d. It is jointly controlled, and Remark 3.9 routes it to a joint surface in $\mathcal{D}_B$. When the constituent artifacts are already registered, that outcome is an omitted-*surface* finding under Definition 7.5, a gap in the coverage of the evaluation rather than a gap in the disclosure of artifacts. It becomes an omitted-mechanism finding only when a controlling artifact is absent from D_B.

Proposition 7.9 (Omitted Use Refutes Completeness and Overturns a Pass)**.** *An omitted-use witness for (d, B, U_d) refutes the use-completeness claim for (d, B, U_d). If the audit returned* PASS *for d on (R, H) under U_d, then the audit over $U_d^* = U_d \frown \langle u^* \rangle$ returns* FAIL*, and (r, s) is a divergence witness for it.*

Proof. The witness exhibits an identity-relevant use of d within B that is absent from U_d, which refutes the claim. By clause (b) the use factors through d, so $\left(d, U_d^*\right)$ satisfies Definition 3.8(c) and remains an audit surface. Under U_d^* the signature of r extends by $u^*(r)$ and that of s by $u^*(s)$, which differ, so $\mathsf{IT}_d(r) \neq \mathsf{IT}_d(s)$ while $r \sim_\tau s$. □

The three completeness claims are independent, and the audit is relative to all of them. Registry completeness can hold while family completeness fails, and the mechanism responsible is then one the operator disclosed and the auditor never evaluated. Registry and family completeness can both hold while use completeness fails, and the surface responsible is then one the audit examined and cleared.

Remark 7.10 (What Licenses the System-Level Reading). Proposition 4.6 combines the surface verdicts into the store operational partition π_B. That composition is sound over the examined family A_B and over each U_d. Reading π_B as the rule of sameness the system implements, rather than as the rule of sameness the evaluated surfaces and their identified uses implement, requires three claims:

(1) registry completeness, that D_B contains every controlling artifact within B;
(2) family completeness, that A_B contains a surface through whose mechanism every identity-relevant use factors when its controlling artifacts lie in D_B; and
(3) use completeness for each $d \in A_B$, that U_d contains every identity-relevant use factoring through d.

None is established by the procedure. Each is refutable by a finite witness, and none is provable from the artifact that asserts it.

7.3 Scopes of a Finding

Audit findings therefore have five distinct scopes. A divergence witness for a surface in A_B identifies a fault in the examined implementation, classified by Definition 4.8. An omitted-mechanism witness identifies an incomplete account of the implementation boundary. An omitted-surface witness identifies an incomplete evaluation of artifacts already disclosed, and by Proposition 7.6 it can overturn a store-level passing verdict without any artifact having been withheld. An omitted-use witness identifies an incomplete account of a surface already examined, and by Proposition 7.9 it can overturn a passing verdict on that surface. A passing verdict establishes only that

no disclosed use of any examined surface splits a declared co-reference class on the examined records under the examined history, and Corollary 5.6 bounds even that.

8 Related Work

8.1 Entity Resolution and Record Linkage

Record linkage asks whether two records describe the same real-world entity, and the probabilistic framework of Fellegi and Sunter formalized the decision as a likelihood-ratio test over agreement patterns between record fields (Fellegi and Sunter 1969). The field has developed extensive machinery for similarity measures, classification, and evaluation (Elmagarmid et al. 2007; Christen 2012), for blocking and filtering schemes that partition records into candidate groups before pairwise comparison (Papadakis et al. 2020), and for the scaling and quality challenges of resolution at practical volume (Getoor and Machanavajjhala 2012).

The resemblance to the present result is real and worth stating plainly. A blocking key is a mechanism whose value determines whether two records will be compared, and it therefore induces a partition of the record domain. A blocking key that separates true matches is a mechanism carrying an identity distinction its designer did not intend, which is structurally the fault this paper isolates.

Three differences separate the settings.

The comparison in entity resolution is against ground truth. Records are labeled as matches or non-matches by an external standard, and the matching function is evaluated for error against those labels. The present result has no ground truth and asks for none. It compares two artifacts the system itself supplies: what it declares and what it does. A divergence witness is a disagreement internal to the system, not an error against an outside label, and it is available to an auditor who has no view on which records truly co-refer.

Co-reference is derived differently. Entity resolution derives it from attribute similarity across a pair of records considered on their own. The declared partition here is derived from a transformation history: records co-refer when a chain of transformations connects them, each of whose families is non-breaking under the declared basis. Two records with identical attributes may fail to co-refer when no

history connects them, and two records with quite different attributes may co-refer when a chain of content-preserving transformations does.

The object under evaluation differs. Entity resolution evaluates a matching function. This paper evaluates a partition induced by an implementation mechanism against a partition induced by a declared identity regime, and diagnoses their divergence against an inventory of identity bases. Where entity resolution asks how well a matcher performs, this paper asks which rule of sameness a system is running.

8.2 Provenance and Data Lineage

PROV-DM provides a general model of entities, activities, agents, derivations, generations, uses, associations, and attributions (Moreau and Missier 2013). It supplies the relations needed to identify transformation occurrences and their evidentiary basis.

A derivation relation does not determine whether source and result represent the same referent. A complete provenance history may record that one artifact was derived from another without stating whether the transformation preserved or broke identity under the system's chosen basis. The present result uses family-labeled histories to derive the declared partition, and then compares that partition with behavior. The question is not only what happened, but which rule of sameness the system operationally applied.

8.3 Identity Conditions and Ontology Engineering

Foundational ontologies provide categories for entities, processes, qualities, roles, and related distinctions (Arp et al. 2015; Gangemi et al. 2002; Masolo et al. 2003). Formal ontology and OntoClean examine ontological commitment, identity conditions, rigidity, dependence, and the quality of taxonomic choices (Guarino 1998 1999; Guarino and Welty 2002; Guizzardi 2005).

Those approaches support careful selection of what kinds of things a system represents and which conditions govern their identity. The problem here begins after that selection. The declared identity condition may be sound while workflow or application logic implements another, and the operational identity partition is the object that makes the second condition visible.

8.4 AI-System Identity and Governance

Ferrario develops a trustworthiness-based metaphysics of artificial intelligence systems in which AI-system kinds are fixed by techno-function and synchronic and diachronic identity are governed by trustworthiness profiles and compatible trustworthiness levels (Ferrario 2025). Applied to high-risk systems under the European AI Act, the account makes explicit that lifecycle governance presupposes judgments about whether an updated or separately deployed system remains the same system for regulatory purposes. Ferrario argues that the Act supplies no internal, auditable criterion for synchronic identity and proposes a correspondence map and minimal decision flow for applying the function-plus-trustworthiness criterion in audit and dispute settings (Ferrario 2026a).

A subsequent category-theoretic account fixes an AI-system type datum consisting of a techno-function, trustworthiness profile, and trustworthiness-level function. It distinguishes equality of trustworthiness level, directed trustworthiness-preserving reachability, mutual reachability as state isomorphism, and natural isomorphism of realized system histories (Ferrario 2026b). That account provides a formal hierarchy of weaker and stronger identity criteria, while leaving their detailed recognition in operational governance and MLOps settings to further methodology.

These works make one substantive AI identity criterion explicit, inspectable, and formally structured. The present result addresses a different audit object. It neither selects the correct substantive identity basis nor determines whether trustworthiness is the appropriate basis for AI-system identity. It takes the effective declared regime as an input and asks whether disclosed implementation surfaces induce identity-relevant treatment faithful to that declaration. Its output is therefore not an AI identity decision flow, but a comparison of declared and operational partitions and, on failure, a finite pair witnessing a distinction made by the implementation that the declaration does not make.

8.5 Classification and Institutional Infrastructure

Classification systems shape the practices and institutions that use them (Bowker and Star 1999). Longino emphasizes the social and institutional conditions under which claims become open to criticism and revision (Longino 1990).

A published classification may differ from the classification implemented by the

system. The operational identity partition converts that institutional concern into a finite computed object for one class of stability-critical distinctions.

8.6 Versioning, Digital Twins, and Persistent Objects

Version-control systems, temporal databases, and digital-twin architectures distinguish states, revisions, derivations, and evolving objects. Digital-twin work emphasizes continuity between physical or operational systems and their changing digital representations (Grieves and Vickers 2017; Voas et al. 2025).

These systems may preserve detailed histories while leaving the identity criterion implicit. Version succession does not by itself determine referent identity. Section 6 shows a version counter carrying an identity criterion finer than any basis the system's own inventory supplies, which is a case the versioning literature has the machinery to express and no standing reason to look for.

8.7 Contradiction Detection

Recent accountability protocols for adversarial supply chains show the value of signed event claims, append-only causal histories, and contradiction proof objects. ECO/CPO-DAG (Cochinescu 2026), for example, treats contradiction detection as a supplemental validation layer rather than a truth-establishing consensus mechanism, and compiles a finite, self-verifying object that binds two signed claims to the violated rule.

The present result shares that commitment to inspectable finite witnesses rather than scores. The approaches differ along an axis the protocol itself marks: *a party that never contradicts itself is invisible to contradiction detection.*

A system that consistently applies an undeclared identity rule for what counts as the same lot, the same consignment, the same rule, or the same regulated entity produces no contradiction and emits no contradiction proof object. Contradiction detection finds inconsistent claims within a declared rule system. The divergence witness finds a rule of sameness the system never declared.

8.8 Provenance and Accountability for AI Agents

Recent work argues that accountability for AI agents requires structural provenance rather than isolated component logs. Hu et al. position explicit provenance across the agentic lifecycle as necessary for assigning responsibility when harm emerges from compositions that no single party designed (Hu et al. 2026). Ojewale, Suresh, and Venkatasubramanian propose audit trails as a tamper-evident, system-agnostic ledger linking technical provenance to governance records (Ojewale et al. 2026). PROV-AGENT extends W3C PROV to capture prompts, responses, decisions, and agent interactions across end-to-end workflows (Souza et al. 2025). Work on auditable agents treats auditability as the system property that makes accountability possible (Nian et al. 2026).

These approaches expose the provenance an audit needs. The operational identity partition supplies a complementary question: whether the co-reference and persistence behavior the system exhibits follows the identity basis it declares.

Otsuka, Toyoda, and Leung define AI identity through correspondence between what an agent is declared to be and what it is observed to do (Otsuka et al. 2026). Their declaration-observation gap is closely related. The present result isolates one formal instance of it, gives the observed side a computable form, and classifies the gap against an inventory of identity bases.

8.9 Distinct Contribution

The contribution is the conjunction of five elements:

(1) the operational identity partition, the rule of sameness a deployed system applies, made computable over a disclosed mechanism registry;

(2) a surface-control condition under which a difference in identity treatment is attributable to a named mechanism, together with the record-identifier exclusion that keeps the test from firing on every correct system;

(3) faithfulness as a refinement relation between declared and operational partitions, with a finite witness refuting it, available for every regime in the inventory;

(4) a four-way lattice classification of divergence, when the sibling is informative inside the declared classes, separated from the global condition of regime

substitution that it does not imply, with a finite version-field construction occupying the sub-sibling case;

(5) non-monotonicity of the passing verdict under extension of the transformation history alone, which indexes a passing audit to a history rather than to a store.

The audit is disclosure-relative at three distinct levels: the artifacts represented in D_B, the surfaces included in A_B, and the identity-relevant uses identified in each U_d. Each level carries a completeness claim that the procedure cannot discharge and a finite witness that refutes it.

9 Limits

The result is bounded in nine ways.

First, the audit is relative to the registry. It evaluates disclosed or discovered mechanisms and does not prove that no further identity-relevant mechanism exists outside the examined boundary or will arise in future behavior. Definition 7.2 states the object whose later discovery refutes a completeness claim, and Remark 3.2 states why the registry constraint cannot be dropped.

Second, the audit is relative to the surfaces it evaluated. Registry completeness does not entail that every surface over the registered artifacts was placed in A_B. A use jointly controlled by two registered fields may factor through neither alone, so the surface carrying it may split a declared co-reference class while each constituent field, taken alone, splits nothing. The audit may not have included that joint surface in A_B, so every evaluated singleton surface can pass while the omitted joint surface fails. Definition 7.4 states the claim and Definition 7.5 the object that refutes it.

Third, the audit is relative to the identified uses of each evaluated surface. A surface cleared by the procedure may carry an identity-relevant use the audit did not see, and Proposition 7.9 shows that such a use overturns the verdict. The operational partition is computed from disclosed behavior, and it is the rule of sameness the mechanism applies only under the claim of Definition 7.7.

Fourth, the declared partition depends on the accuracy and completeness of the family-labeled history, which Definition 2.10 states as a precondition. An inaccurate or incomplete history prevents the artifact from representing the examined

transformations, and the correct verdict is INDETERMINATE.

Fifth, the classification is relative to the imported inventory, and it is local to the declared co-reference classes. When the sibling also splits an examined declared class, the four cases of Definition 4.8 locate a divergence against that sibling on pairs the declaration merges, and say nothing about pairs it separates. Otherwise the divergence is unpositioned. Sibling alignment does not establish regime substitution, by Proposition 4.14, and no case of the classification names the basis the implementation carries. Identifying that basis remains work for the auditor.

Sixth, faithfulness is one-directional by design. A surface that merges records the declaration separates is not reported as a fault, for the reason given in Remark 4.2. Conflation of declared-distinct referents at the level of the store is a real failure and is outside the scope of a single-surface audit.

Seventh, verdicts are indexed to (R, H). Proposition 5.5 shows that a passing verdict can be overturned by extending the history alone, with the record domain held fixed. A surface faithful on the examined artifact may diverge on a larger one, and the classification of a divergence may change with it.

Eighth, the procedure decides no downstream substantive question. It does not choose the correct identity basis, and either sibling may be appropriate for a given system. It does not determine causal truth, normative correctness, legal interpretation, legal authority, responsibility, or institutional legitimacy. It preserves a prior condition for examining those questions: stable and inspectable reference to what the claims concern.

Ninth, the worked application is illustrative rather than empirical. The paper does not evaluate the audit against a deployed record system. Such an evaluation would test the practical adequacy of the disclosed registry, examined surfaces, identified uses, and record-indexed encodings, rather than the finite decidability result established here.

10 Conclusion

A record system answers the co-reference question twice: once in what it declares and once in what it does. The declared answer partitions the records into co-reference classes. The implementation supplies a second partition through its

identity-relevant treatment of those records. This paper formalizes that second answer as the operational identity partition and compares it with the declared partition.

The comparison is a refinement test. A mechanism is faithful when it splits no declared co-reference class, and a single divergence witness refutes faithfulness. Faithfulness composes across the examined surface family: a failure on any surface refutes store faithfulness, while the store passes only when every examined surface is faithful.

When faithfulness fails and the sibling basis also splits an examined declared class, the divergence is classified against that alternative basis in one of four sibling-relative cases. Otherwise the divergence is unpositioned. The classification is local to the declared co-reference classes. Alignment with the sibling inside those classes does not establish that the system implements the sibling regime, and a single witness does not establish alignment. A version counter incremented on every textual edit splits declared classes strictly more finely than either imported basis. The worked version-field example therefore inhabits the sub-sibling case.

The audit is decidable, three-valued, and disclosure-relative at three distinct levels: the artifacts represented in D_B, the surfaces included in A_B, and the identity-relevant uses identified in each U_d. Each level carries a completeness claim that the procedure cannot discharge and a finite witness that refutes it.

A passing verdict is also history-relative. Because declared co-reference is a transitive closure, extending the history alone, with the record domain and operational treatment unchanged, can merge declared classes and create a witness among records already examined. A passing audit is a statement about a history, not a certificate about a store.

Neutral Substrates states the constraint on shared foundational commitments. *Referential Regimes* states how identity persists under transformation. Both describe what a system should declare. The present result supplies the other half of the comparison: a computable account of the identity relation induced by the disclosed implementation, together with explicit limits on how far that account reaches.

Statements and Declarations

Acknowledgments

The author thanks the anonymous reviewers of earlier papers in this series for comments that improved the framing, organization, and presentation of this work. This research made use of SciX, a scientific literature search and discovery platform, whose related-literature tools helped identify relevant work across disciplinary boundaries.

Author Contributions

The author is the sole contributor to this work and is responsible for all aspects of the research, authorship, and publication.

Use of AI-Assisted Tools

AI-assisted tools were used for editing, formatting, and consistency checking. The author reviewed all suggestions and is solely responsible for the content.

Declaration of Conflicting Interest

The author declares no potential conflicts of interest.

References

R. Arp, B. Smith, and A. D. Spear. *Building Ontologies with Basic Formal Ontology*. MIT Press, 2015.

G. C. Bowker and S. L. Star. *Sorting Things Out: Classification and Its Consequences*. MIT Press, Cambridge, MA, 1999.

D. M. Case. *Neutral Substrates: A Design Constraint for Shared Records Under Persistent Interpretive Disagreement*. arXiv preprint arXiv:2601.14271, 2026.

D. M. Case. *Referential Regimes: Transformation-Invariant Identity for Neutral Substrates*. arXiv preprint arXiv:2601.16152, 2026.

P. Christen. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer, Berlin, 2012.

S. Cochinescu. *ECO/CPO-DAG: A Contradiction-Based Accountability Layer for Adversarial Supply Chains*. arXiv preprint arXiv:2607.06804, 2026.

A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Verykios. Duplicate record detection: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 19(1):1–16, 2007.

I. P. Fellegi and A. B. Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969.

A. Ferrario. A trustworthiness-based metaphysics of artificial intelligence systems. In *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency (FAccT '25)*, pages 1360–1370. ACM, 2025. doi:10.1145/3715275.3732091. arXiv:2506.03233.

A. Ferrario. *High-Risk AI Systems and the Problem of Identity in the European AI Act*. arXiv preprint arXiv:2605.23922, 2026.

A. Ferrario. *A Category Theory Account of AI Identity*. arXiv preprint arXiv:2607.00220, 2026.

A. Gangemi, N. Guarino, C. Masolo, A. Oltramari, and L. Schneider. Sweetening ontologies with DOLCE. In *Knowledge Engineering and Knowledge Management: Ontologies and the Semantic Web (EKAW 2002)*, volume 2473 of LNCS, pages 166–181. Springer, 2002.

L. Getoor and A. Machanavajjhala. Entity resolution: Theory, practice, and open challenges. *Proceedings of the VLDB Endowment*, 5(12):2018–2019, 2012.

M. Grieves and J. Vickers. Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems. In F.-J. Kahlen, S. Flumerfelt, and A. Alves, editors, *Transdisciplinary Perspectives on Complex Systems*, pages 85–113. Springer, 2017.

N. Guarino. Formal ontology and information systems. In N. Guarino, editor, *Formal Ontology in Information Systems: Proceedings of FOIS '98*, pages 3–15. IOS Press, 1998.

N. Guarino. The role of identity conditions in ontology design. In C. Freksa and D. M. Mark, editors, *Spatial Information Theory: Cognitive and Computational Foundations of Geographic Information Science (COSIT '99)*, volume 1661 of LNCS, pages 221–234. Springer, 1999. doi:10.1007/3-540-48384-5_15.

N. Guarino and C. A. Welty. Evaluating ontological decisions with OntoClean. *Communications of the ACM*, 45(2):61–65, 2002.

G. Guizzardi. *Ontological Foundations for Structural Conceptual Models*. PhD thesis, University of Twente, 2005.

J. Hu, X. Huang, Q. He, Y. Sun, Y. Dong, and X. Huang. *Responsible Agentic AI Requires Explicit Provenance*. arXiv preprint arXiv:2605.17169, 2026.

N. Kolt. *Superintelligence and Law. Harvard Journal of Law & Technology*, forthcoming, 2026. arXiv:2603.28669.

H. E. Longino. *Science as Social Knowledge: Values and Objectivity in Scientific Inquiry*. Princeton University Press, 1990.

C. Masolo, S. Borgo, A. Gangemi, N. Guarino, and A. Oltramari. *WonderWeb Deliverable D18: Ontology Library*. IST Project 2001-33052 WonderWeb, 2003.

L. Moreau and P. Missier, editors. *PROV-DM: The PROV Data Model*. W3C Recommendation, 30 April 2013.

Y. Nian, A. Yuan, H. Zhang, J. Li, and Y. Zhao. *Auditable Agents*. arXiv preprint arXiv:2604.05485, 2026.

V. Ojewale, H. Suresh, and S. Venkatasubramanian. *Audit Trails for Accountability in Large Language Models*. arXiv preprint arXiv:2601.20727, 2026.

T. Otsuka, K. Toyoda, and A. Leung. *AI Identity: Standards, Gaps, and Research Directions for AI Agents*. arXiv preprint arXiv:2604.23280, 2026.

G. Papadakis, D. Skoutas, E. Thanos, and T. Palpanas. Blocking and filtering techniques for entity resolution: A survey. *ACM Computing Surveys*, 53(2), 2020.

R. Souza, A. Gueroudji, S. DeWitt, D. Rosendo, T. Ghosal, R. Ross, P. Balaprakash, and R. Ferreira da Silva. PROV-AGENT: Unified provenance for tracking AI agent interactions in agentic workflows. In *Proceedings of the 2025 IEEE 21st International Conference on e-Science*, pages 467–473. Chicago, IL, 2025. doi:10.1109/eScience65000.2025.00093. arXiv:2508.02866.

L. Staufer, K. Feng, K. Wei, L. Bailey, Y. Duan, M. Yang, A. P. Ozisik, S. Casper, and N. Kolt. The 2025 AI Agent Index: Documenting technical and safety features of deployed agentic AI systems. In *Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency (FAccT '26)*, pages 1536–1576. ACM, 2026. doi:10.1145/3805689.3806728.

J. Voas, P. Mell, P. Laplante, and V. Piroumian. *Security and Trust Considerations for Digital Twin Technology.* NIST Internal Report (IR) 8356. National Institute of Standards and Technology, Gaithersburg, MD, 2025. doi:10.6028/NIST.IR.8356.